

CO4

ASSESSMENT TOOL-2

NAME : P. Ranjithkumar

DATE:13.08.2026

REG.NO: 192411119

Thursday

COURSE CODE : CSA1103

COURSE NAME : OOAD

1. Legacy E-Learning Management System

(Reverse Engineering using Object-Oriented Principles and Layered Architecture)

1. Introduction

A **Legacy E-Learning Management System (LMS)** is used to manage student registration, course management, content delivery, assignment evaluation, and result generation. In the existing system, the **User Interface (UI), business logic, and database operations are tightly coupled**, making the application difficult to modify, maintain, and extend. Reverse engineering helps identify the existing components and redesign them into a modular object-oriented architecture.

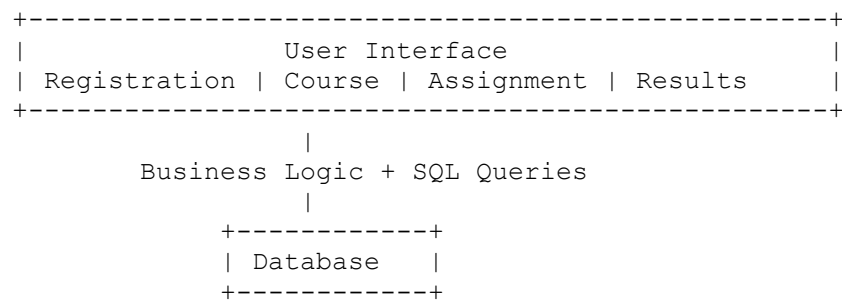
2. Existing System Analysis

The legacy system consists of the following major modules:

- Student Registration
- Course Management
- Content Delivery
- Assignment Evaluation
- Result Generation

These modules directly access the database, and all business rules are embedded within the UI code.

Existing Architecture



3. Problems in the Existing System

- **Tight Coupling:** UI, business logic, and database code are combined.
- **Low Cohesion:** A single class performs multiple unrelated tasks.
- **Code Duplication:** Validation and SQL queries are repeated.
- **Poor Maintainability:** Small changes affect multiple modules.
- **Limited Scalability:** Adding new features requires significant code modifications.
- **Difficult Testing:** Components cannot be tested independently.

4. Reverse Engineering

The following domain classes can be identified after analyzing the existing system.

Class	Responsibility
Student	Register courses and submit assignments
Course	Manage course information
Instructor	Upload learning materials
Content	Store and deliver study materials
Assignment	Manage assignment submission and evaluation
Result	Calculate and publish grades
Registration	Handle student enrollment

5. Attributes and Methods

Student Class

Attributes

- studentId
- studentName
- email
- department

Methods

- registerCourse()
- submitAssignment()
- viewResult()

Course Class

Attributes

- courseId
- courseName
- credits

Methods

- addCourse()
- updateCourse()
- removeCourse()

Instructor Class

Attributes

- instructorId
- instructorName

Methods

- uploadContent()
- createAssignment()

- evaluateAssignment()

Assignment Class

Attributes

- assignmentId
- title
- deadline

Methods

- submit()
- evaluate()
- calculateMarks()

Result Class

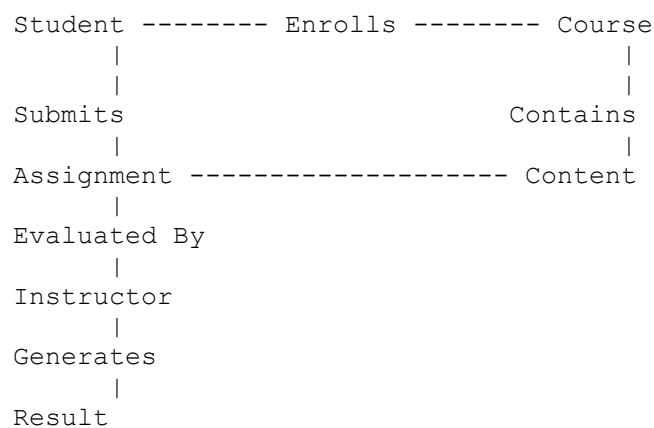
Attributes

- resultId
- marks
- grade

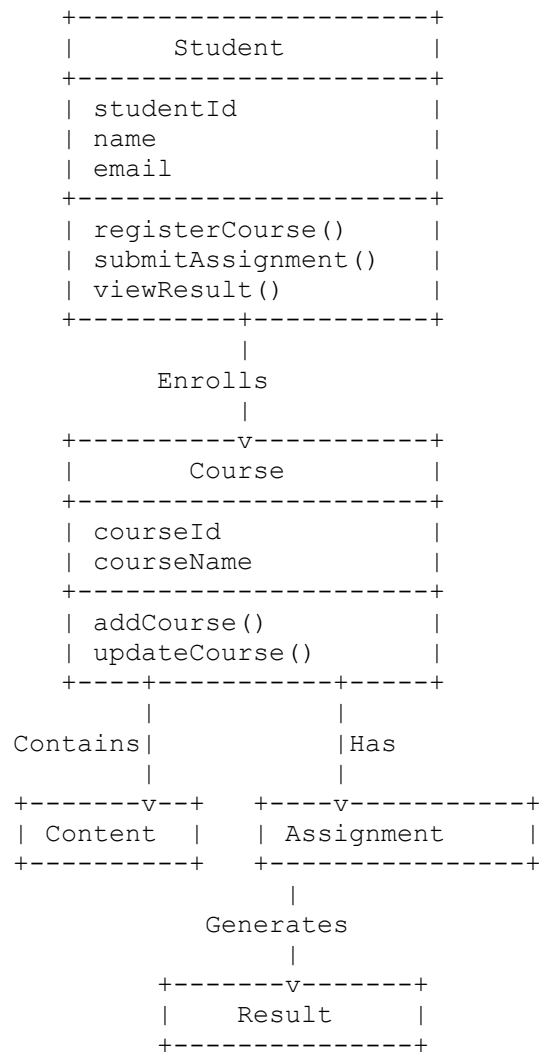
Methods

- generateResult()
- publishResult()

6. Relationships Between Classes

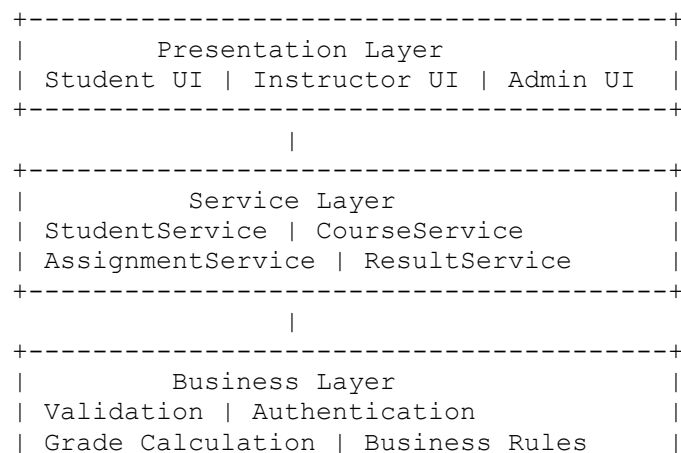


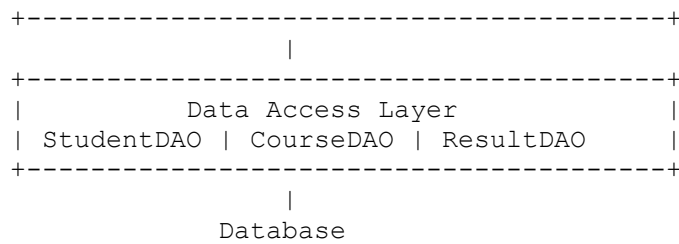
7. UML Class Diagram



8. Redesigned Layered Architecture

The redesigned system follows a **4-layer architecture**.





9. Object-Oriented Principles Applied

Encapsulation

Each class keeps its own data and methods together.

Example:

- Student class stores student details and registration methods.

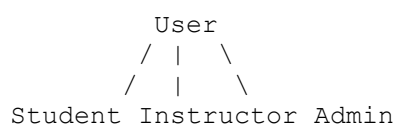
Abstraction

Only essential functions are visible to users.

Example:

- `registerCourse()`
- `viewResult()`

Inheritance



Common properties are inherited from the `User` class.

Polymorphism

Different user types can implement the `login()` method differently.

10. Improvements After Redesign

Quality Attribute Legacy System Redesigned System

Cohesion	Low	High
Coupling	Tight	Loose

Quality Attribute	Legacy System	Redesigned System
Maintainability	Difficult	Easy
Scalability	Poor	High
Code Reuse	Low	High
Testing	Difficult	Easy

11. Advantages

- Clear separation of UI, business logic, and database.
- High cohesion by assigning a single responsibility to each class.
- Loose coupling through layered architecture.
- Easier maintenance and debugging.
- Better scalability for adding new features.
- Reusable classes reduce code duplication.
- Improved readability and modularity.
- Simplified testing of individual layers.

12. Conclusion

The legacy E-Learning Management System has tightly coupled components, making maintenance and future enhancements difficult. By reverse engineering the system and redesigning it using **Object-Oriented Principles** and **Layered Architecture**, the application becomes more **modular, reusable, maintainable, and scalable**. The redesigned system improves software quality by increasing **cohesion**, reducing **coupling**, and providing a clear separation between presentation, business, and data access layers. This architecture supports future enhancements, such as online examinations, discussion forums, and mobile learning, with minimal changes to the existing code.

2. Legacy Customer Support System

(Reverse Engineering using Behavioral Design Pattern)

1. Introduction

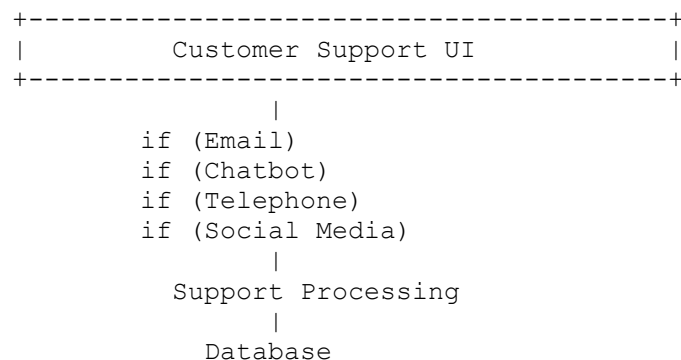
A **Customer Support System** is used to handle customer requests through different communication channels such as **Email, Chatbot, Telephone, and Social Media**. In the legacy system, multiple **if-else** or **switch-case** statements are used to process each support request. Whenever a new support channel is introduced, developers must modify several existing modules, making the system difficult to maintain and extend.

The system can be redesigned using the **Strategy Design Pattern** (Behavioral Pattern), where each support channel has its own implementation. This improves flexibility, extensibility, and maintainability.

2. Existing System Analysis

The legacy application processes customer requests using conditional statements.

Existing Architecture



3. Problems in the Existing System

- **Large if-else statements** make the code difficult to read.
- **Tight coupling** between support channels and business logic.
- **Violation of Open/Closed Principle (OCP)** because adding a new channel requires modifying existing code.
- **Poor maintainability** due to repeated code.
- **Limited scalability** when introducing new communication channels.
- **Difficult testing** because all logic is in one class.

4. Reverse Engineering

The following classes can be identified from the existing implementation.

Class	Responsibility
Customer	Sends support request
SupportRequest	Stores customer issue
EmailSupport	Handles email requests
ChatbotSupport	Handles chatbot requests
TelephoneSupport	Handles phone requests

Class	Responsibility
SocialMediaSupport	Handles social media requests
SupportManager	Selects the appropriate support channel

5. Attributes and Methods

Customer Class

Attributes

- customerId
- customerName
- email

Methods

- createRequest()
- viewStatus()

SupportRequest Class

Attributes

- requestId
- issue
- priority
- status

Methods

- submitRequest()
- updateStatus()

SupportManager Class

Methods

- setSupportChannel()
- processRequest()

Support Channel Classes

EmailSupport

- processRequest()

ChatbotSupport

- processRequest()

TelephoneSupport

- processRequest()

SocialMediaSupport

- processRequest()

6. Existing Processing Logic

```
Customer Request
|
if (Email)
|
Email Processing

else if (Chatbot)
|
Chatbot Processing

else if (Telephone)
|
Telephone Processing

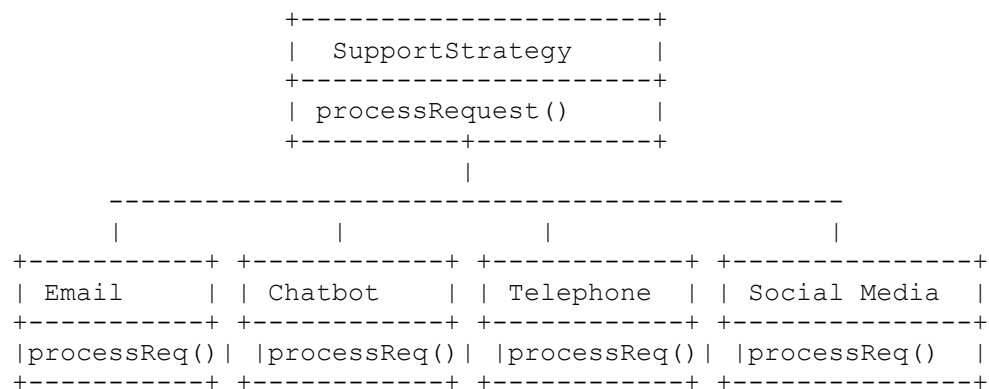
else if (Social Media)
|
Social Media Processing
```

Adding a new channel (e.g., WhatsApp) requires modifying the above code.

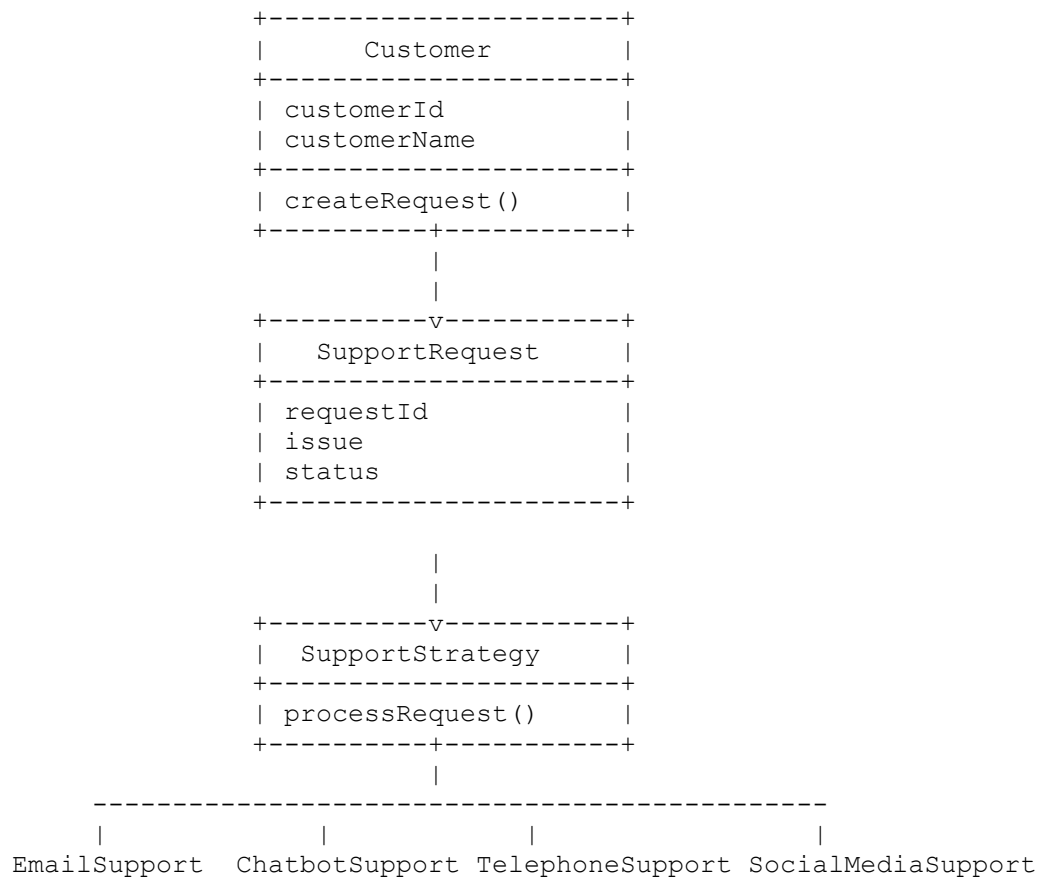
7. Redesigned Solution Using Strategy Design Pattern

Instead of conditional statements, each support channel implements a common interface.

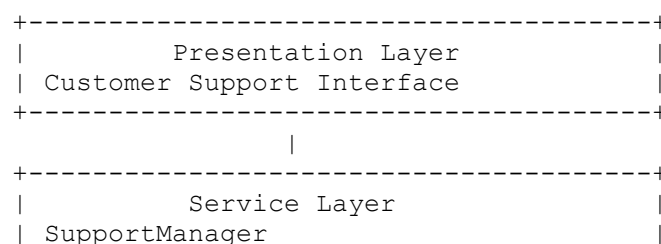
Strategy Pattern Diagram

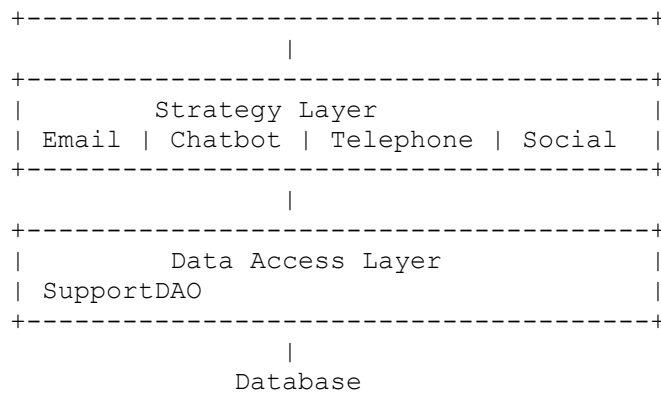


8. UML Class Diagram



9. Redesigned Architecture





10. Improvements After Redesign

Quality Attribute Legacy System Redesigned System

Flexibility	Low	High
Extensibility	Poor	Excellent
Coupling	Tight	Loose
Maintainability	Difficult	Easy
Scalability	Limited	High
Code Reuse	Low	High

11. Advantages

- Eliminates long **if-else** statements.
- New support channels can be added without modifying existing classes.
- Improves **Open/Closed Principle (OCP)**.
- Reduces code duplication.
- Enhances flexibility through the Strategy Pattern.
- Easier testing because each support channel can be tested independently.
- Better readability and maintainability.
- Supports future communication channels (e.g., WhatsApp, SMS, Mobile App) with minimal code changes.

12. Conclusion

The legacy Customer Support System suffers from tightly coupled code and extensive conditional statements, making it difficult to extend and maintain. By reverse engineering the system and redesigning it using the **Strategy Design Pattern**, each support channel becomes an independent component implementing a common interface. This approach improves

flexibility, extensibility, loose coupling, maintainability, code reuse, and scalability, resulting in a cleaner and more robust customer support application.

3. Legacy Banking Transaction System

(Reverse Engineering using Layered Architecture and Design Patterns)

1. Introduction

A **Banking Transaction System** is used to manage customer accounts, validate transactions, perform fund transfers, process loans, and generate transaction reports. In the legacy system, all these operations are implemented inside a few large classes, and database operations are written directly within business methods. This results in **tight coupling**, poor maintainability, and difficulty in testing.

The system can be redesigned by identifying **domain classes, service classes, and data access components**, and by applying **Layered Architecture, DAO (Data Access Object), and Service Layer** design patterns.

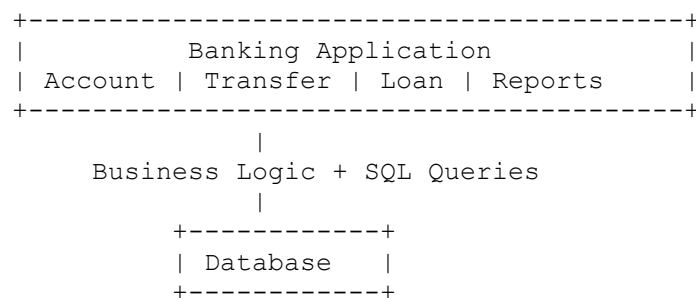
2. Existing System Analysis

The legacy banking system contains the following modules:

- Account Management
- Transaction Validation
- Fund Transfer
- Loan Processing
- Transaction Reporting

These modules directly communicate with the database.

Existing Architecture



3. Problems in the Existing System

- Business logic and database operations are mixed together.
- Tight coupling between application and database.
- Low cohesion because one class performs multiple tasks.
- Repeated SQL queries in different modules.
- Difficult to maintain and test.
- Adding new banking services requires major code changes.

4. Reverse Engineering

The following classes are identified after analyzing the existing system.

Class	Responsibility
Customer	Maintain customer information
Account	Store account details
Transaction	Process deposits and withdrawals
FundTransfer	Transfer money between accounts
Loan	Manage loan processing
Report	Generate transaction reports
AccountService	Execute business rules
TransactionDAO	Perform database operations

5. Attributes and Methods

Account Class

Attributes

- accountNumber
- accountHolder
- balance
- accountType

Methods

- deposit()
- withdraw()

- getBalance()

Transaction Class

Attributes

- transactionId
- amount
- transactionDate

Methods

- validateTransaction()
- processTransaction()

Loan Class

Attributes

- loanId
- loanAmount
- interestRate

Methods

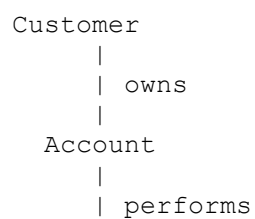
- applyLoan()
- approveLoan()

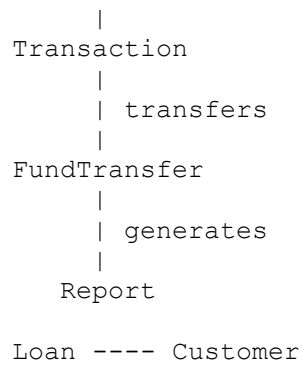
Report Class

Methods

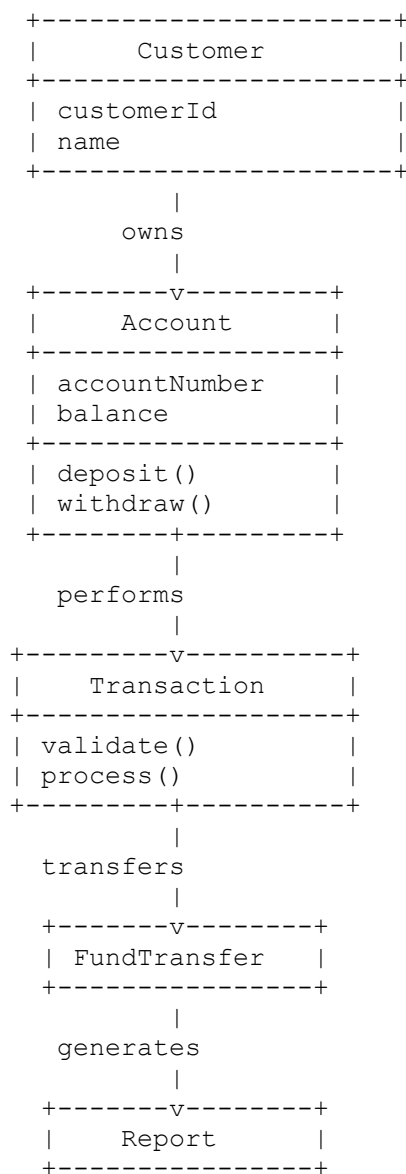
- generateReport()
- printStatement()

6. Relationships Between Classes



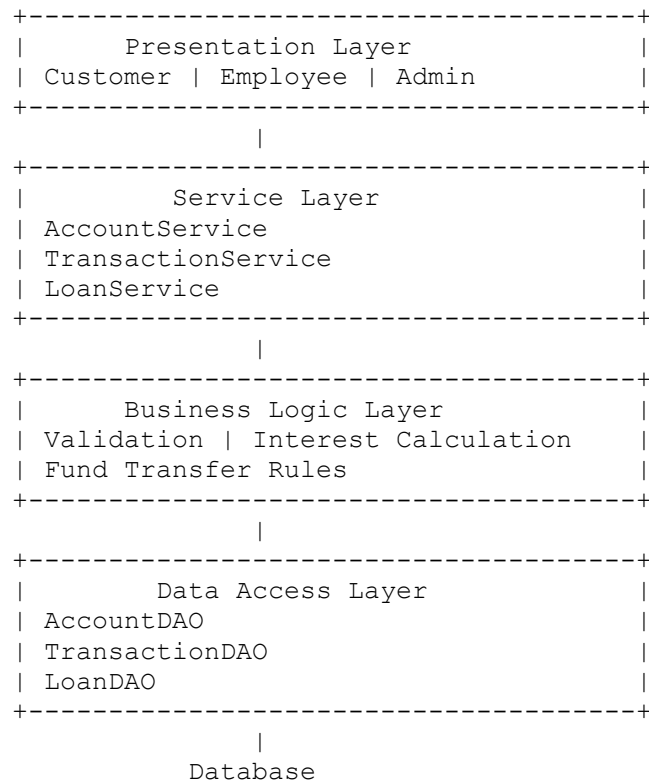


7. UML Class Diagram



8. Redesigned Layered Architecture

The redesigned application separates the system into four layers.



9. Design Patterns Used

1. DAO (Data Access Object)

Separates database operations from business logic.

Example:

- AccountDAO
- TransactionDAO
- LoanDAO

2. Service Layer Pattern

Handles business rules separately.

Example:

- AccountService
- TransactionService
- LoanService

3. MVC / Layered Architecture

Separates:

- Presentation Layer
- Business Layer
- Data Layer

This improves modularity.

10. Improvements After Redesign

Quality Attribute	Legacy System	Redesigned System
Separation of Concerns	Poor	Excellent
Coupling	Tight	Loose
Cohesion	Low	High
Maintainability	Difficult	Easy
Testability	Low	High
Scalability	Poor	Excellent

11. Advantages

- Clear separation between business logic and database operations.
- DAO pattern reduces duplicate SQL code.
- High cohesion with each class having a single responsibility.
- Loose coupling through service and DAO layers.
- Easier unit testing of business logic.
- Better scalability for adding new banking features.
- Improved code reusability and maintainability.
- Faster debugging and easier future enhancements.

12. Conclusion

The legacy Banking Transaction System combines account management, transaction processing, loan management, reporting, and database operations into a few tightly coupled classes, making the system difficult to maintain and extend. By reverse engineering the system and redesigning it using **Domain Classes**, **Service Layer**, **DAO Pattern**, and **Layered Architecture**, the application achieves better **separation of concerns**, **high cohesion**, **loose coupling**, **improved testability**, **scalability**, and **maintainability**. The redesigned architecture provides a reliable and efficient solution for modern banking applications.

4. Legacy Hotel Reservation System

(Reverse Engineering using DAO/Repository Pattern and Layered Architecture)

1. Introduction

A **Hotel Reservation System** is used to manage **room booking**, **guest details**, **reservations**, **payments**, and **room availability**. In the legacy system, reservation screens directly perform **OODBMS (Object-Oriented Database Management System)** operations. The same database queries are repeated across different modules, causing **high coupling**, **code duplication**, and **poor persistence management**.

The system can be redesigned using the **DAO (Data Access Object)** or **Repository Pattern** with a **Layered Architecture**, separating database operations from business logic to improve modularity and maintainability.

2. Existing System Analysis

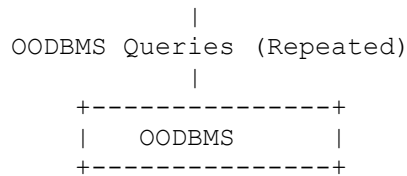
The legacy hotel reservation application contains:

- Guest Management
- Room Management
- Reservation Management
- Payment Processing

Each screen directly communicates with the OODBMS.

Existing Architecture

```
+-----+
|      Hotel Reservation Screens      |
| Guest | Room | Booking | Payment   |
+-----+
```



3. Problems in the Existing System

- UI directly accesses the database.
- Repeated OODBMS queries across multiple modules.
- Tight coupling between UI and persistence layer.
- Poor code reusability.
- Difficult maintenance when database schema changes.
- Low cohesion because one class performs multiple functions.
- Difficult testing of reservation logic.

4. Reverse Engineering

The following classes are identified from the legacy system.

Class	Responsibility
Guest	Store guest information
Room	Maintain room details
Reservation	Handle room booking
Payment	Process payments
ReservationService	Business logic for reservations
RoomDAO	Access room data
GuestDAO	Manage guest records
ReservationDAO	Handle reservation data
PaymentDAO	Store payment information

5. Attributes and Methods

Guest Class

Attributes

- guestId

- guestName
- phone
- email

Methods

- registerGuest()
- updateGuest()

Room Class

Attributes

- roomId
- roomType
- roomPrice
- roomStatus

Methods

- checkAvailability()
- updateRoom()

Reservation Class

Attributes

- reservationId
- checkInDate
- checkOutDate

Methods

- reserveRoom()
- cancelReservation()

Payment Class

Attributes

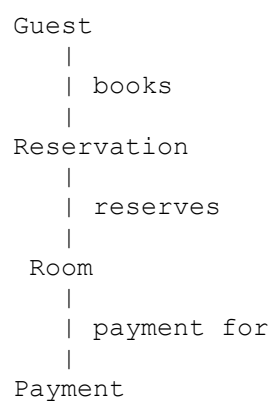
- paymentId

- amount
- paymentStatus

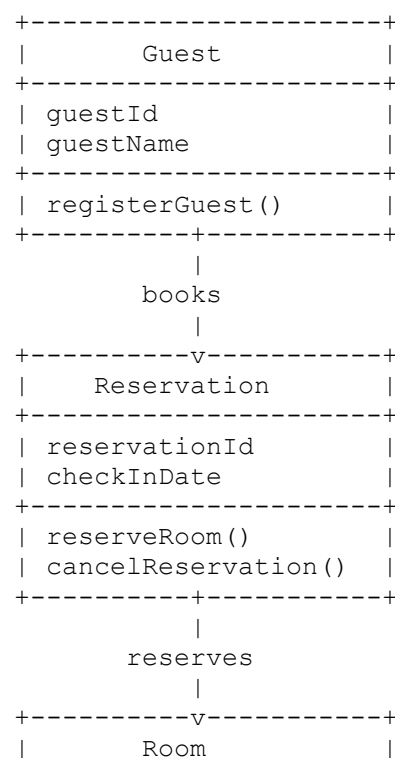
Methods

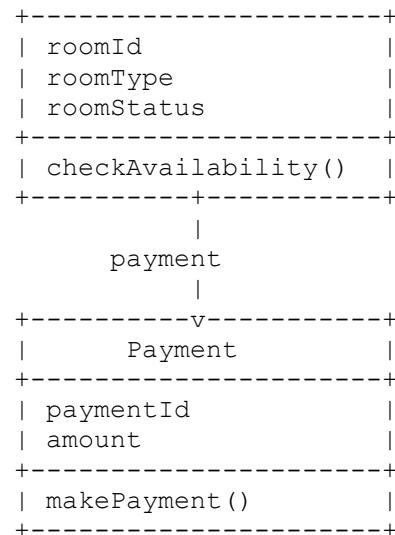
- makePayment()
- generateReceipt()

6. Class Relationship Diagram



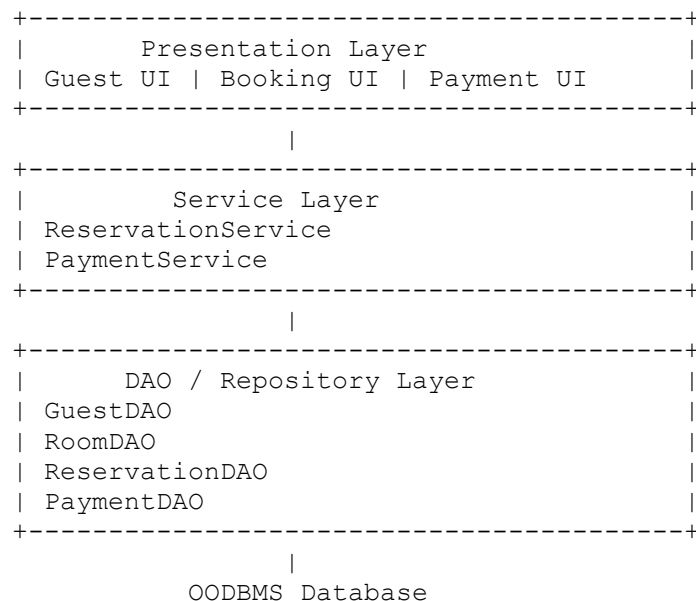
7. UML Class Diagram





8. Redesigned Layered Architecture

The redesigned system separates presentation, business logic, and persistence.



9. DAO / Repository Pattern

The **DAO (Data Access Object)** pattern separates database operations from business logic.

DAO Components

- **GuestDAO** → Guest database operations

- **RoomDAO** → Room availability operations
- **ReservationDAO** → Booking records
- **PaymentDAO** → Payment transactions

This removes repeated database queries from the user interface.

10. Improvements After Redesign

Quality Attribute	Legacy System	Redesigned System
Coupling	Tight	Loose
Code Duplication	High	Low
Reusability	Poor	High
Modularity	Low	High
Maintainability	Difficult	Easy
OODBMS Integration	Poor	Efficient

11. Advantages

- Database operations are centralized in DAO classes.
- Reduces duplicate OODBMS queries.
- Loose coupling between UI and database.
- High cohesion by assigning a single responsibility to each class.
- Easier maintenance when database changes occur.
- Better modularity and code reusability.
- Simplified testing using DAO and service layers.
- Improved scalability for adding new hotel services such as online check-in or loyalty programs.

12. Conclusion

The legacy Hotel Reservation System suffers from direct database access, repeated OODBMS queries, and tightly coupled components. By reverse engineering the application and redesigning it using the **DAO/Repository Pattern** and **Layered Architecture**, database access is separated from business logic, resulting in **high cohesion, loose coupling, improved modularity, better code reuse, easier maintenance, and efficient OODBMS integration**. This redesigned architecture provides a scalable and maintainable solution for modern hotel reservation systems.

5. Legacy Transportation Management System

(Reverse Engineering using Factory Method / Abstract Factory Design Pattern)

1. Introduction

A **Transportation Management System (TMS)** is used to manage different transportation services such as **buses, trains, taxis, and rental vehicles**. In the legacy system, object creation for each transportation type is duplicated across different modules. Whenever a new transportation service is added, developers must modify multiple classes, resulting in poor maintainability and scalability.

The system can be redesigned using the **Factory Method** or **Abstract Factory** design pattern to centralize object creation and improve extensibility.

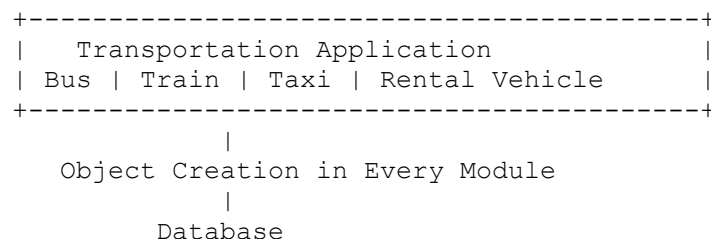
2. Existing System Analysis

The legacy transportation application contains:

- Bus Module
- Train Module
- Taxi Module
- Rental Vehicle Module

Each module creates its own objects independently.

Existing Architecture



3. Problems in the Existing System

- Object creation code is repeated in many modules.
- Tight coupling between application logic and object creation.
- Difficult to introduce a new transportation service.
- Low code reusability.

- Poor scalability.
- Violates the **Open/Closed Principle (OCP)**.

4. Reverse Engineering

The following classes are identified in the legacy system.

Class	Responsibility
Vehicle	Common transportation interface
Bus	Bus transportation service
Train	Train transportation service
Taxi	Taxi booking service
RentalVehicle	Rental vehicle management
TransportFactory	Creates transportation objects
BookingService	Handles bookings

5. Attributes and Methods

Vehicle (Abstract Class)

Attributes

- vehicleId
- vehicleType
- fare

Methods

- book()
- cancel()
- calculateFare()

Bus Class

Methods

- book()
- calculateFare()

Train Class

Methods

- book()
- calculateFare()

Taxi Class

Methods

- book()
- calculateFare()

RentalVehicle Class

Methods

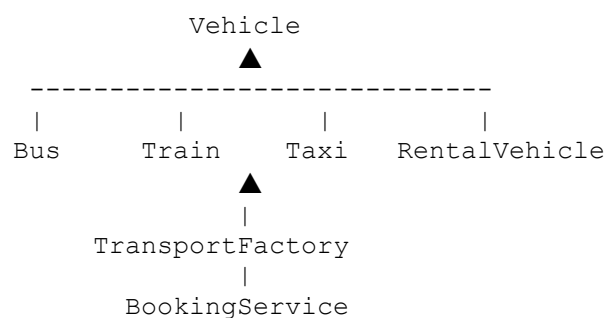
- book()
- calculateFare()

TransportFactory Class

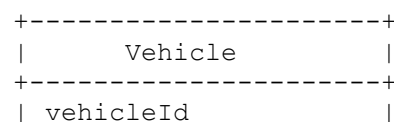
Methods

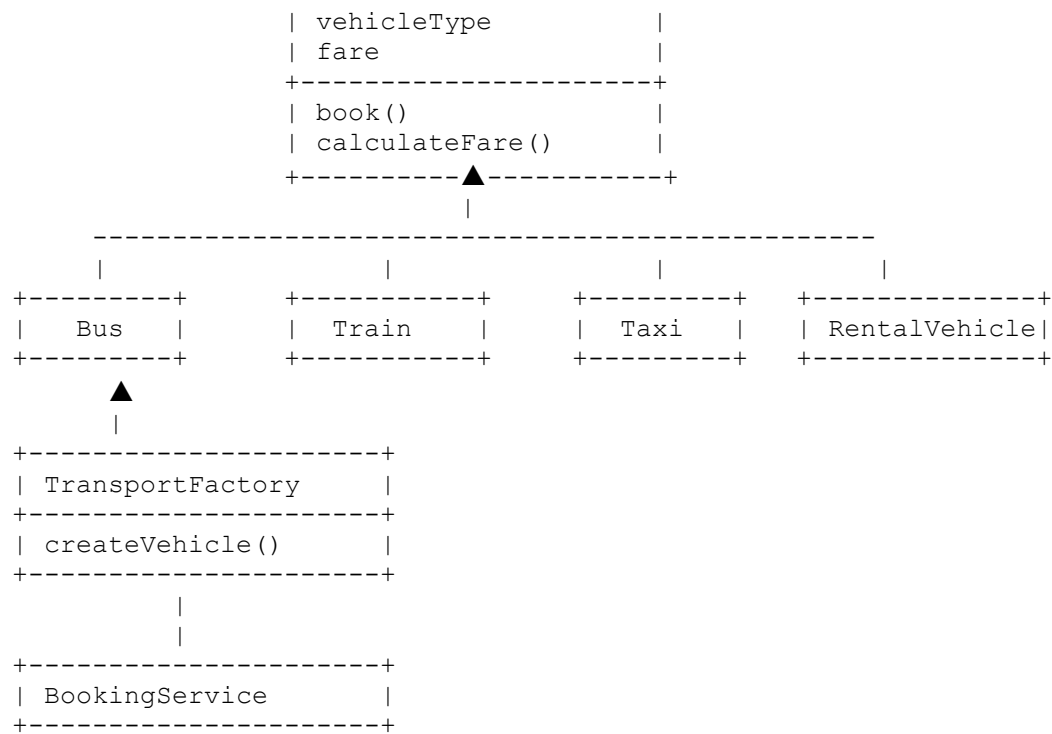
- createVehicle()

6. Class Relationship Diagram



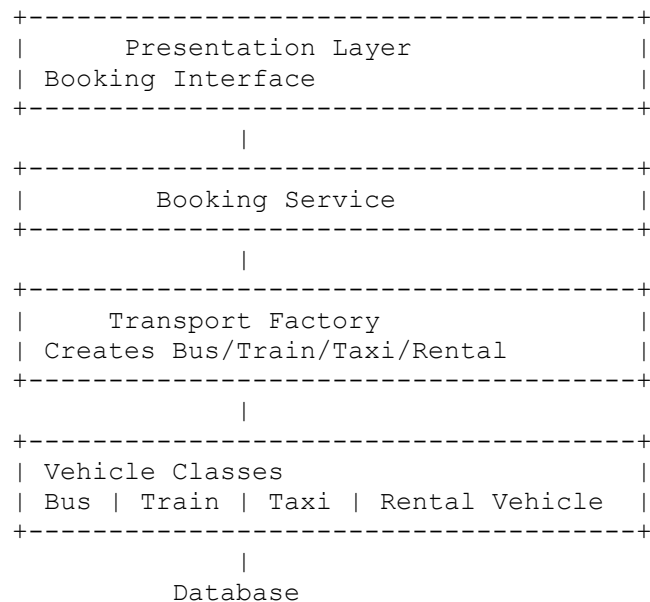
7. UML Class Diagram





8. Redesigned Architecture Using Factory Method

The object creation process is moved to a factory class.



9. Factory Method Design Pattern

The **Factory Method Pattern** creates transportation objects without exposing the object creation logic to the client.

Example:

- **TransportFactory**
 - createVehicle("Bus")
 - createVehicle("Train")
 - createVehicle("Taxi")
 - createVehicle("RentalVehicle")

If a new transportation type such as **Metro** or **Flight** is added, only the factory needs to be updated without affecting existing booking logic.

10. Improvements After Redesign

Quality Attribute	Legacy System	Redesigned System
Object Creation	Repeated	Centralized
Coupling	Tight	Loose
Code Reuse	Low	High
Extensibility	Poor	Excellent
Scalability	Limited	High
Maintainability	Difficult	Easy

11. Advantages

- Centralized object creation using the Factory Method.
- Eliminates duplicate object creation code.
- Loose coupling between client code and vehicle classes.
- Easy to introduce new transportation services.
- High code reusability.
- Improved scalability and maintainability.
- Supports future expansion without modifying existing business logic.
- Easier testing because object creation is separated from application logic.

12. Conclusion

The legacy Transportation Management System contains duplicated object creation logic and tightly coupled modules, making it difficult to extend and maintain. By reverse engineering the system and redesigning it using the **Factory Method (or Abstract Factory) Design Pattern**, object creation is centralized and separated from business logic. The redesigned system improves **extensibility, scalability, loose coupling, code reuse, and maintainability**, making it easier to support additional transportation services such as metro, flights, or bike rentals in the future.

6. Legacy Healthcare Monitoring System

(Reverse Engineering using SOLID Principles and Design Patterns)

1. Introduction

A **Healthcare Monitoring System** collects patient information, monitors sensor readings, generates emergency alerts, stores medical records, and produces health reports. In the legacy system, a few classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components. This violates several **SOLID principles**, making the application difficult to maintain and extend.

The system can be redesigned using **interfaces, abstractions, dependency injection, and design patterns** to improve flexibility, maintainability, and testability.

2. Existing System Analysis

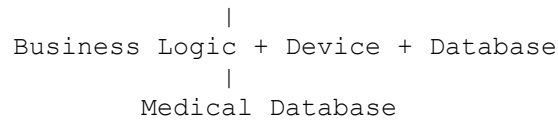
The legacy system contains the following modules:

- Patient Information
- Sensor Monitoring
- Alert Generation
- Medical Record Storage
- Report Generation

All these functions are handled by a few large classes.

Existing Architecture

```
+-----+
|      Healthcare Monitoring System      |
| Patient | Sensor | Alerts | Reports   |
+-----+
```



3. Problems in the Existing System

- One class performs multiple unrelated tasks.
- Direct dependency on specific sensor devices.
- Database code mixed with business logic.
- Difficult to add new monitoring devices.
- Low cohesion and tight coupling.
- Difficult to test and maintain.

4. Violations of SOLID Principles

1. Single Responsibility Principle (SRP)

A single class manages patient data, sensor readings, reports, and alerts.

Violation: One class has multiple responsibilities.

2. Open/Closed Principle (OCP)

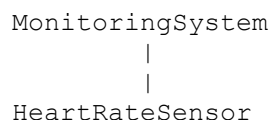
Adding a new medical sensor requires modifying existing classes.

Violation: Existing code must be changed whenever a new device is introduced.

3. Dependency Inversion Principle (DIP)

The monitoring class directly depends on concrete sensor implementations.

Example:



Instead, it should depend on an interface.

5. Reverse Engineering

The following classes are identified.

Class	Responsibility
Patient	Store patient information
Sensor	Read health data
Alert	Generate emergency alerts
MedicalRecord	Store patient records
Report	Generate medical reports
MonitoringService	Process sensor data
StorageService	Save records

6. Attributes and Methods

Patient Class

Attributes

- patientId
- patientName
- age
- bloodGroup

Methods

- registerPatient()
- updateDetails()

Sensor Class

Attributes

- sensorId
- sensorType

Methods

- readData()
- sendReading()

Alert Class

Methods

- generateAlert()
- notifyDoctor()

MedicalRecord Class

Methods

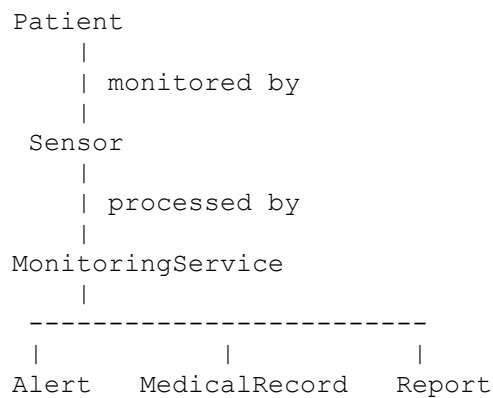
- saveRecord()
- retrieveRecord()

Report Class

Methods

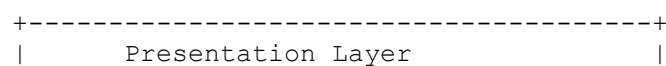
- generateReport()
- printReport()

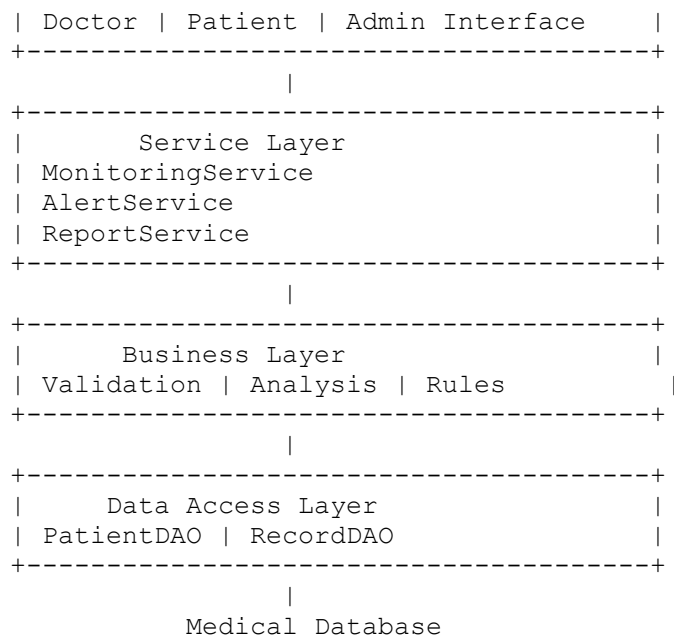
7. Class Relationship Diagram



8. Redesigned Architecture

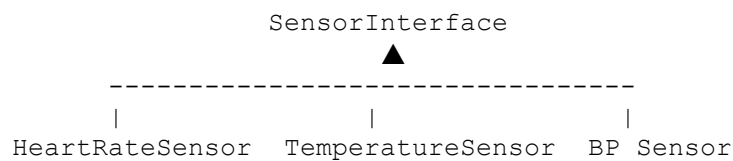
The redesigned system follows a layered architecture.





9. Using Interfaces (DIP)

Instead of depending on a concrete sensor, create an interface.



Now, the monitoring service communicates with the **SensorInterface**, allowing new devices to be added without changing existing code.

10. Design Patterns Used

1. Strategy Pattern

Used to support different sensor-processing algorithms.

Example:

- Heart Rate Monitoring
- Temperature Monitoring
- Blood Pressure Monitoring

Each sensor uses its own processing strategy.

2. Dependency Injection (DIP)

The monitoring service receives sensor objects through interfaces instead of creating them directly.

3. DAO Pattern

Separates medical record storage from business logic.

Example:

- PatientDAO
- MedicalRecordDAO

11. Improvements After Redesign

Quality Attribute	Legacy System	Redesigned System
Cohesion	Low	High
Coupling	Tight	Loose
Flexibility	Poor	High
Testability	Difficult	Easy
Maintainability	Low	High
Scalability	Limited	Excellent

12. Advantages

- High cohesion through single-responsibility classes.
- Loose coupling using interfaces and dependency injection.
- Easy to add new monitoring devices without modifying existing code.
- Better flexibility using the Strategy Pattern.
- Improved maintainability with separate service and DAO layers.
- Easier unit testing due to modular design.
- Reusable components reduce code duplication.
- Supports future enhancements such as wearable devices, IoT sensors, and AI-based health analysis.

13. Conclusion

The legacy Healthcare Monitoring System suffers from multiple SOLID principle violations, including **SRP**, **OCP**, and **DIP**, resulting in tightly coupled classes and poor maintainability. By reverse engineering the system and redesigning it using **interfaces**, **layered architecture**, **Dependency Injection**, **Strategy Pattern**, and **DAO Pattern**, the application becomes **more cohesive, loosely coupled, flexible, testable, and scalable**. The redesigned architecture is well-suited for modern healthcare systems that require support for new medical devices, cloud storage, and future technological advancements.